Software Design Specification

<Project Title>

Project Code:
LPSN-0099

Internal Advisor:
Dr. Hassam Ali

External Advisor:
Mr. Irfan Olak

Project Manager:
Prof Dr. Muhammad Ilyaas

Project Team:
Muhammad Rizwan (BSCS51F22S003) Team Lead
Dua Kiran Faraz (BSCS51F22S002)
Waleeja Hafeez (BSCS51F22S049)

Submission Date:
15/12/2025

_____
Project Manager's Signature

Document Information

Category	Information
Customer	UOS – Department of Computer Science
Project	**Localized Professional Service And Product Network**
Document	Software Design Specification
Document Version	1.0
Identifier	LPSN-0099
Status	Draft
Author(s)	Team of this project “LPSN” (Muhammad Rizwan ; Dua Kiran Faraz ; Waleeja hafeez)
Approver(s)	Prof Dr.Muhammad Ilyaas
Issue Date	dec. 15, 2025
Document Location	
Distribution	1. Prof Dr.Muhammad Ilyaas 2. Prof Dr.Hassam Ali 3. Project team

Definition of Terms, Acronyms and Abbreviations

Table of Contents

1. Introduction

1.1 Purpose of Document

This Software Design Specification (SDS) document provides a detailed design view of the Localized Professional Product and Service Network system. The document is intended for project supervisors, developers, evaluators, and future maintainers of the system. It explains the system architecture, design decisions, components, and detailed design models. The project follows an Object-Oriented Design (OOD) methodology using UML diagrams.

1.2 Project Overview

The Localized Professional Product and Service Network is a web and mobile-based platform that connects local professionals, service providers, and product sellers with customers in the same geographic area. The system enables users to search, book, and review services or products while allowing professionals to manage profiles, services, pricing, and availability. The design approach focuses on modularity, scalability, security, and ease of use.

1.3 Scope

In Scope:

1. User registration and authentication
2. Location-based search and filtering
3. Booking and order management
4. Rating and review system
5. Admin dashboard for monitoring and management

Out of Scope:

1. International service availability
2. online transaction handling

2. Design Considerations

2.1 Assumptions and Dependencies

Assumptions:

1. Users (customers and service providers) will cooperate fully in adopting the app and provide accurate input data, including profile information, service requests, and payment details.
2. The app's hardware and software requirements will be met by users' devices, including smartphones, tablets, or laptops with compatible operating systems (e.g., iOS, Android, Windows).
3. Internet connectivity will remain stable for real-time data synchronization.
4. Users will receive basic training on how to use the app effectively, including tutorials, FAQs, and in-app support.
5. Service providers will have the necessary equipment, licenses, and certifications to deliver services.

Dependencies:

1. The app relies on external APIs for location services (e.g., Google Maps, Apple Maps), and other integrations (e.g., SMS gateways, email services).
2. The app depends on a stable connection to the central database for data storage and retrieval, using a cloud-based database service (e.g., AWS RDS, Google Cloud SQL).

3. Regular data updates and maintenance are necessary to ensure accuracy and prevent inconsistencies, including data backups, security patches, and software updates.
4. The app's performance and scalability depend on the underlying infrastructure, including server capacity, network bandwidth, and database optimization.

2.2 Risks and Volatile Areas

1. **Integration with External Systems**

Risk:

Need to integrate with location services, and other third-party APIs. Protocols may change, and APIs may be deprecated or replaced.

Design Response:

- Use API gateway + adapter layer to abstract the integration and provide a standardized interface.
- Defined integration contracts with versioning to ensure compatibility and backward compatibility.
- Decouple external systems with message queues (e.g., RabbitMQ, Apache Kafka) to handle asynchronous communication and reduce dependencies.
- Implement API monitoring and alerting to detect issues and changes in real-time.

2. Technology and Infrastructure Changes

Risk:

Upgrading databases, switching cloud services, or adopting new frameworks can disrupt the system and impact performance, scalability, or security.

Design Response:

- Use containerized deployments (Docker/K8s) to provide a consistent and reproducible environment.
- Abstract data access using repository pattern to decouple the business logic from the data storage.
- Stateless backend services where possible to enable horizontal scaling and load balancing.
- Implement continuous integration and delivery (CI/CD) pipelines to automate testing, building, and deployment.

3. Performance and Scalability Demands

Risk:

Increased user volume, higher concurrency, real-time requests, and heavy computational tasks (e.g., image processing, NLP) can impact performance and scalability.

Design Response:

- Auto-scaling for API services to dynamically adjust capacity based on demand.
- Caching layers for frequent reads (e.g., Redis, Memcached) to reduce database load and improve response times.
- Queue-based processing for heavy modules (e.g., image processing, NLP) to offload computationally intensive tasks.
- Implement load testing and performance monitoring to identify bottlenecks and optimize the system.

4. Security Threats

Risk:

Cyberattacks, unauthorized access, misuse of user data, and data breaches can compromise the system's security and trust.

Design Response:

- Role-based + attribute-based access control to restrict access to sensitive data and functionality.
- Encryption of data in transit (TLS) and at rest (AES-256) to protect user data.
- Continuous monitoring and vulnerability scans to detect and respond to security threats.
- Implement incident response plan and disaster recovery procedures to minimize the impact of a security breach.

5. Third-Party Service Dependency Risks

Risk:

location services, or other third-party services may change pricing, APIs, or availability, impacting the system's functionality and revenue.

Design Response:

- Use abstraction/interface layer to decouple the third-party services from the business logic.
- Circuit breakers and retries to handle failures and reduce the impact of outages.
- Monitor third-party services' status and performance to detect issues and changes.

6. Operational Risks

Risk: Server downtime, network issues, database failure, or other operational issues can impact the system's availability and performance.

Design Response:

- Redundancy: master-slave DB, load balancer, and failover mechanisms to ensure high availability.

- Backup + disaster recovery plan to ensure business continuity in case of a disaster.
- Health monitoring + alerting to detect issues and respond quickly.
- Implement automated testing and validation to ensure the system is functioning correctly.

3 System Architecture

3.0 Use Case diagram

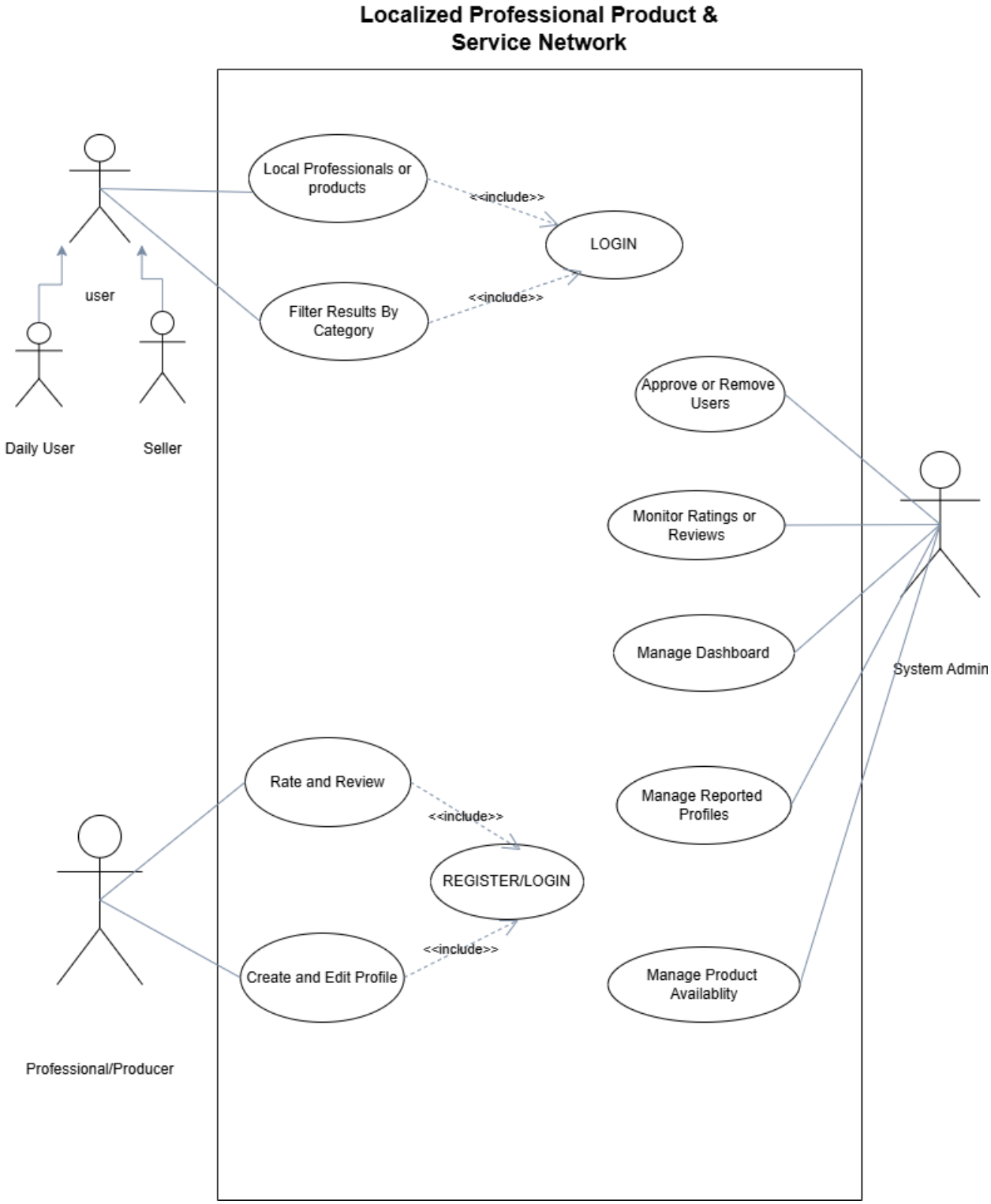

Figure 3.0 Use Case Diagram

https://drive.google.com/file/d/1QG4kVnGefgGn_nG2-GSWAzzpy8aVUF51/view?usp=sharing

3.2 System Level Architecture

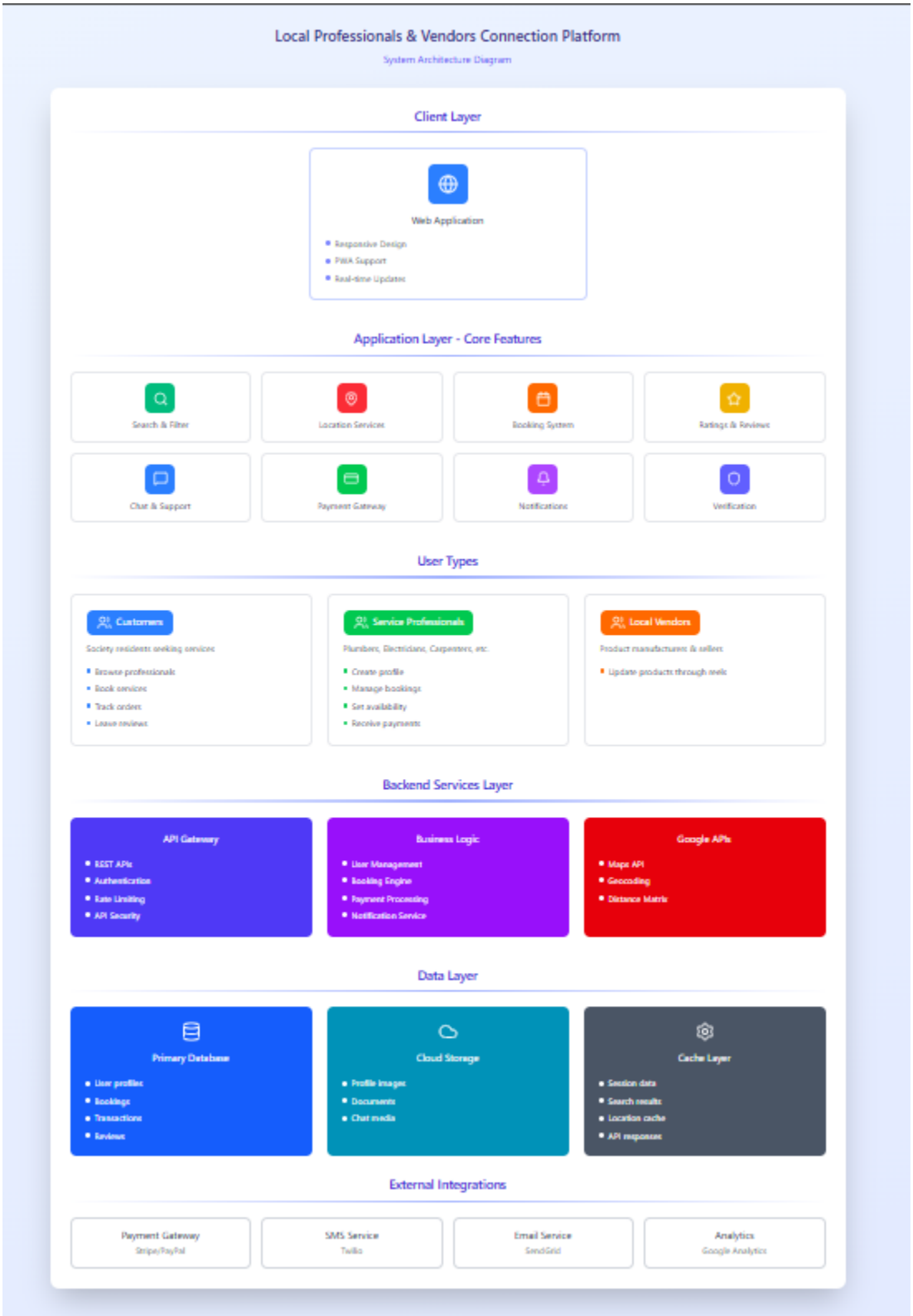

Figure 3.2 Architecture Diagram

https://www.figma.com/make/viEKEK5zDu9upmBkBEdSZa/System-Architecture-Diagram?t=cskBm3l48Vd7mjAC-20&fullscreen=

3.3 Sub-System / Component / Module Level Architecture

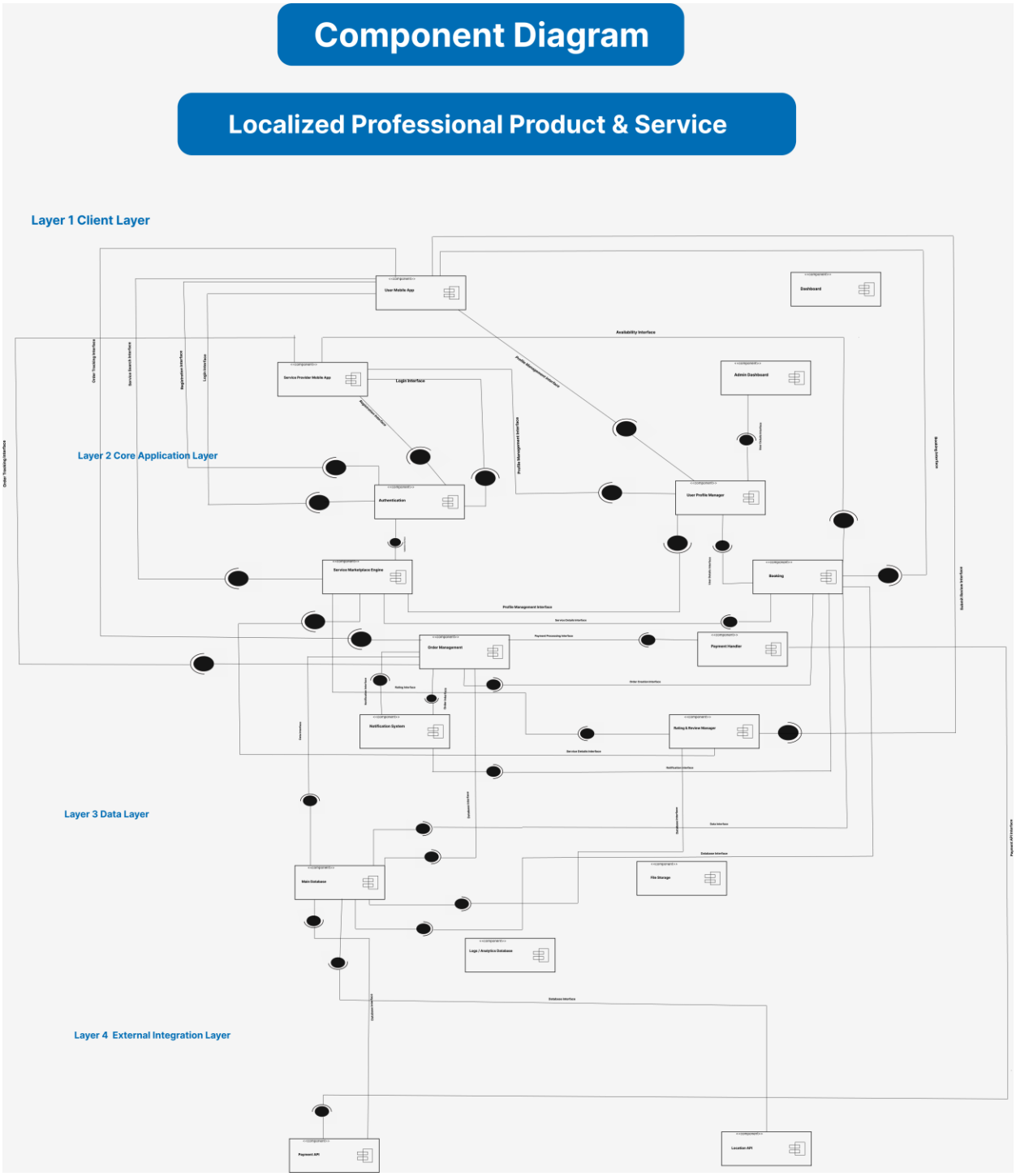

Figure 3.3 Component Diagram

https://www.figma.com/design/9Zyuqgc2tlq2lULMDy8Jb0/Untitled?node-id=0-1&t=sb2h936X1WmHmXZ0-1

3.4 Sub-Component / Sub-Module Level Architecture (1…n)

Identify all the sub components or sub modules (if any) of the already identified modules and components. Provide their diagrammatic view using appropriate detailed architecture diagram presenting how those subsystems, modules and components are further divided into sub components and sub modules and how they interact with each other

4 Design Strategies

4.1 Strategy 1:

Modular Layered Architecture

The system is structured into logical layers:

1. **Presentation Layer (Frontend UI)**
   - Customer app, service provider app, admin dashboard
2. **Application Layer (Backend Logic)**
   - Service booking, order management, payment processing, notification handling
3. **Data Access Layer**
   - Repository classes, ORM queries, database communication
4. **Database Layer**
   - PostgreSQL for structured data, MongoDB for unstructured data (e.g., images, documents)

Reasoning:

- Separates UI, business logic, and database interaction
- Makes each module easy to modify or extend
- Reduces complexity and avoids tangled dependencies

Trade-offs:

- Requires discipline to maintain layer boundaries
- Slight overhead in communication between layers

4.2 Strategy 2:

API-Driven / Service-Oriented Design

All modules communicate using clearly defined REST APIs. Examples:

- /services/book
- /orders/update-status
- /payments/process

Reasoning:

- Clean separation between frontend and backend
- Easy to integrate with external services or third-party apps
- Allows independent scaling of services later

Trade-offs:

- More API design work initially
- Network latency between services

4.3 Strategy 3:

Future System Extension & Plug-in Architecture

The architecture is designed to allow new workflows and modules without rewriting existing ones. Strategies used:
- Interface-based design
- Configurable modules
- Separate routing for each domain (services, orders, payments)

Reasoning:
- Allows easy addition of new services or features
- Reduces impact on existing codebase

Trade-offs:
- Requires abstraction and planning during early design
- Some modules might have more configuration overhead

4.4 Strategy 4: Unified Data Management Strategy

The system uses a consistent approach to data storage:
- PostgreSQL for structured data
- Cloud storage for files (images, documents)
- ACID transactions for sensitive updates
- Foreign keys to prevent inconsistent data
- Indexed queries for real-time performance

Reasoning:
- Ensures correctness and consistency of data
- Enables fast retrieval of data
- Supports large-scale operations

Trade-offs:
- Requires schema planning
- Some features (e.g., full-text search) need extra tools

4.5 Strategy 5: Concurrency & Synchronization Controls

Some actions need safe handling of parallel requests:
- Service booking
- Order updates
- Payment processing
- Simultaneous updates on service provider availability

Concurrency mechanisms used:
- Database locks
- Transaction isolation
- Atomic operations
- Queues for long tasks (payment processing, notification sending)

Reasoning:

- Guarantees consistent data
- Prevents race conditions
- Critical for time-sensitive operations

Trade-offs:
- High concurrency may reduce performance due to locking
- Requires careful backend design

4.6 Strategy 6: Reusability of Components

Common utilities are designed for reuse across modules:
- Logging service
- Authentication middleware
- File upload utility
- Notification/email service
- Validation helpers

Reasoning:
- Reduces repeated code
- Makes maintenance easier
- Ensures consistent behavior across modules

Trade-offs:
- Shared services must be carefully versioned to avoid breaking modules

4.7 Strategy 7: Secure-by-Design Architecture

Security is built into the architecture instead of added later:
- Role-based access control (Customer, Service Provider, Admin, etc.)
- Encrypted communication (HTTPS)
- Input validation
- Audit logging
- Secure authentication tokens

Reasoning:
- Protects sensitive user data
- Meets regulatory requirements (e.g., GDPR, HIPAA)

Trade-offs:
- Adds development overhead
- Requires periodic updates and policy checks.

5 Detailed Design System

5.1 Class Diagram

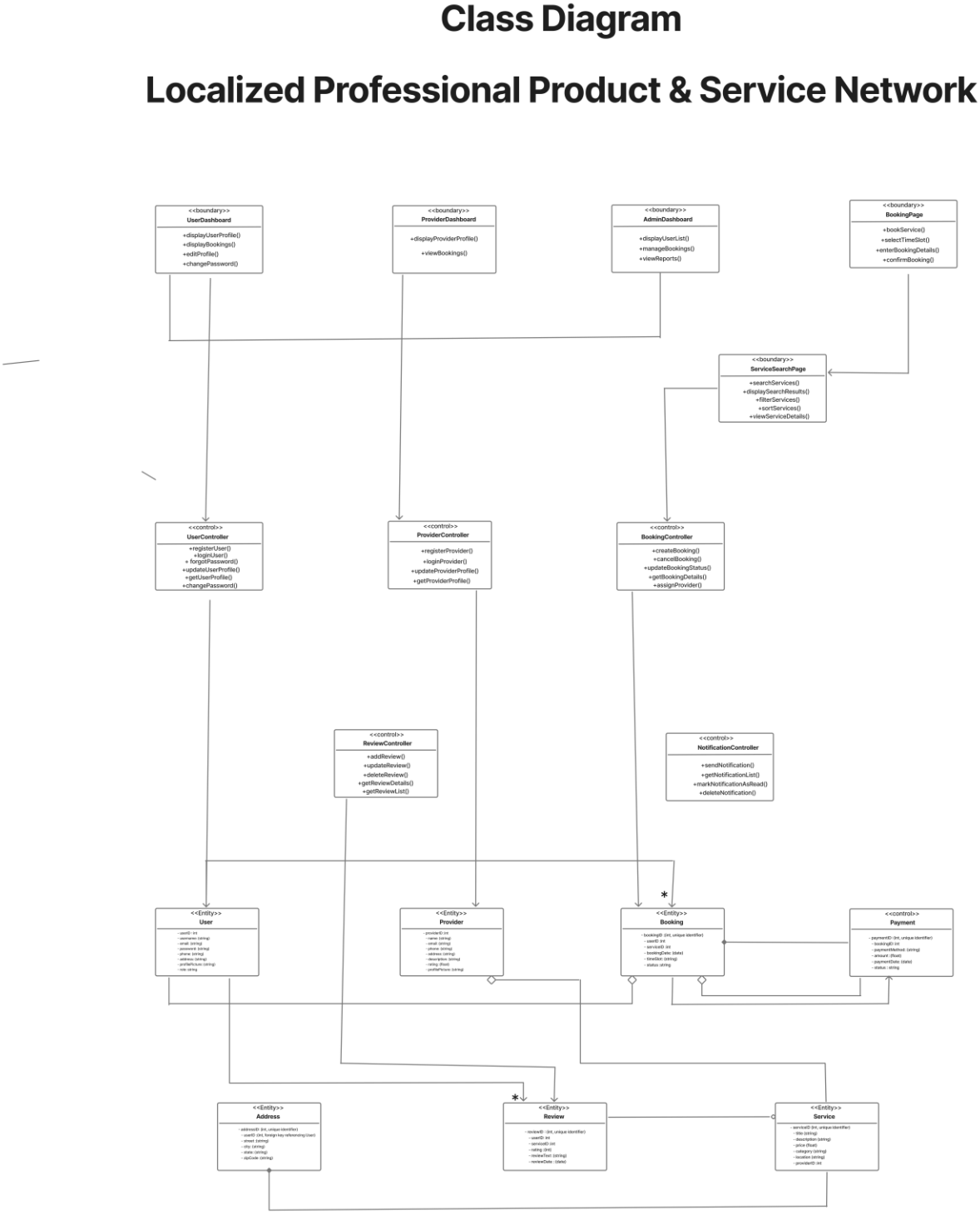

Figure 5.1 Class Diagram

https://www.figma.com/board/vjKK8dWnMWoyBIbrh5Em8H/Untitled?node-id=0-1&t=E7M5zSE3YVfXvZc6-1

5.2 Sequence Diagram

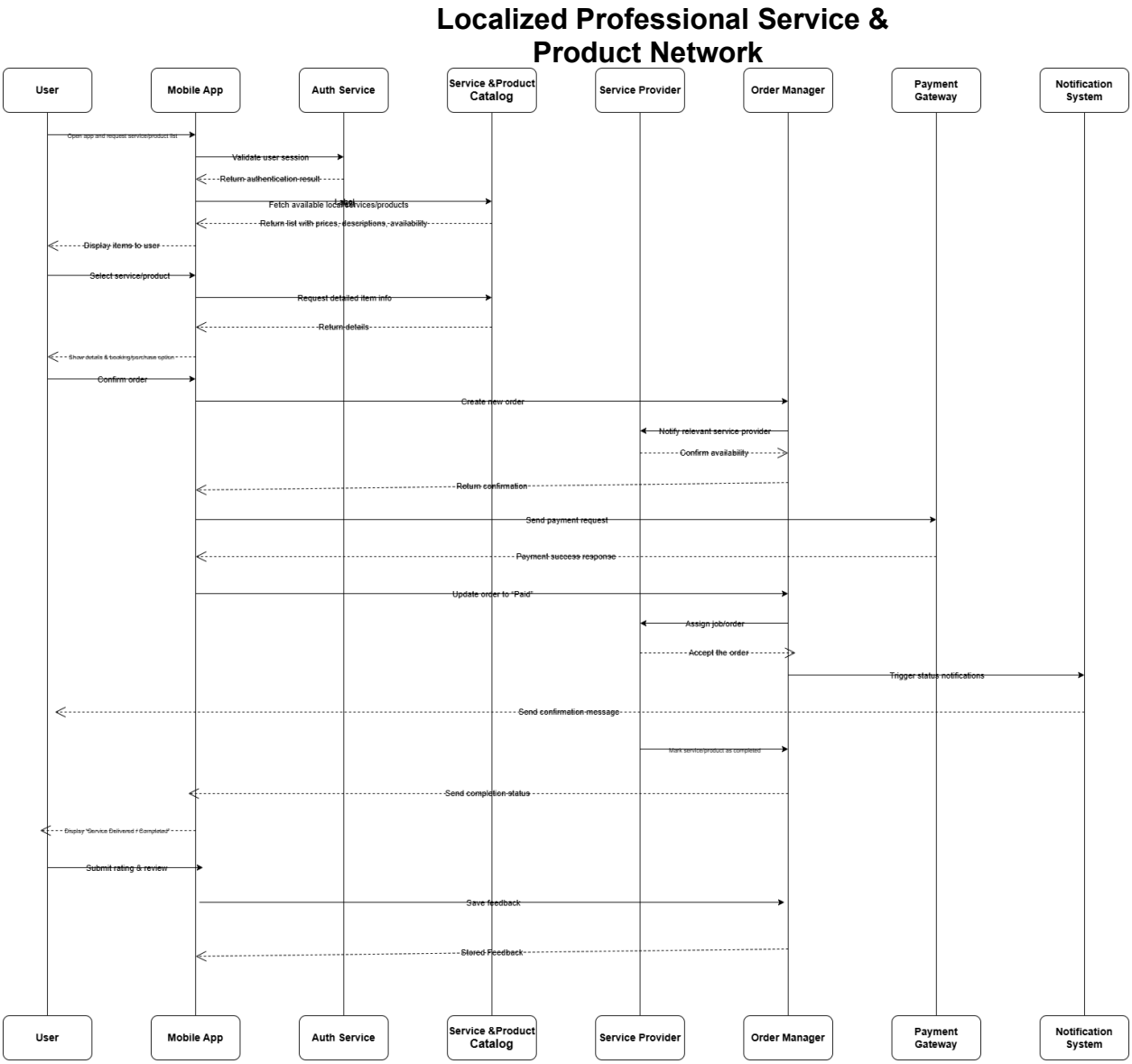

Figure 5.2 Sequence Diagram

https://drive.google.com/file/d/13SePbbJyz5ycmcEvC0ZURzqA4u7-zZsa/view?usp=sharing

5.3 Transition Diagram

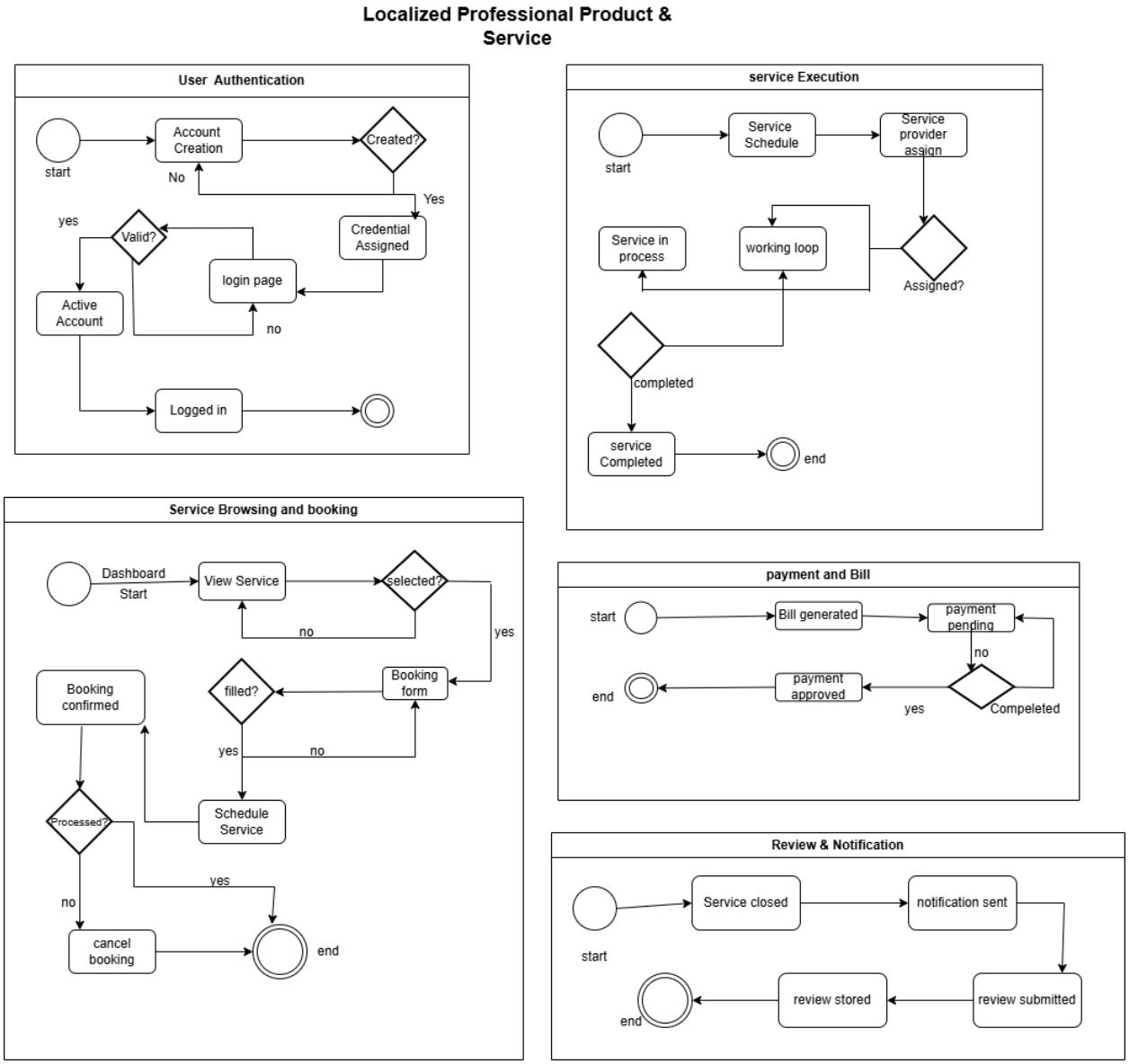

Figure 5.3 state transition diagram

https://drive.google.com/file/d/1nLgwEOmxZz0W35-9P4MOMV1L34V8HSC6/view?usp=sharing

5.4 ER Diagram

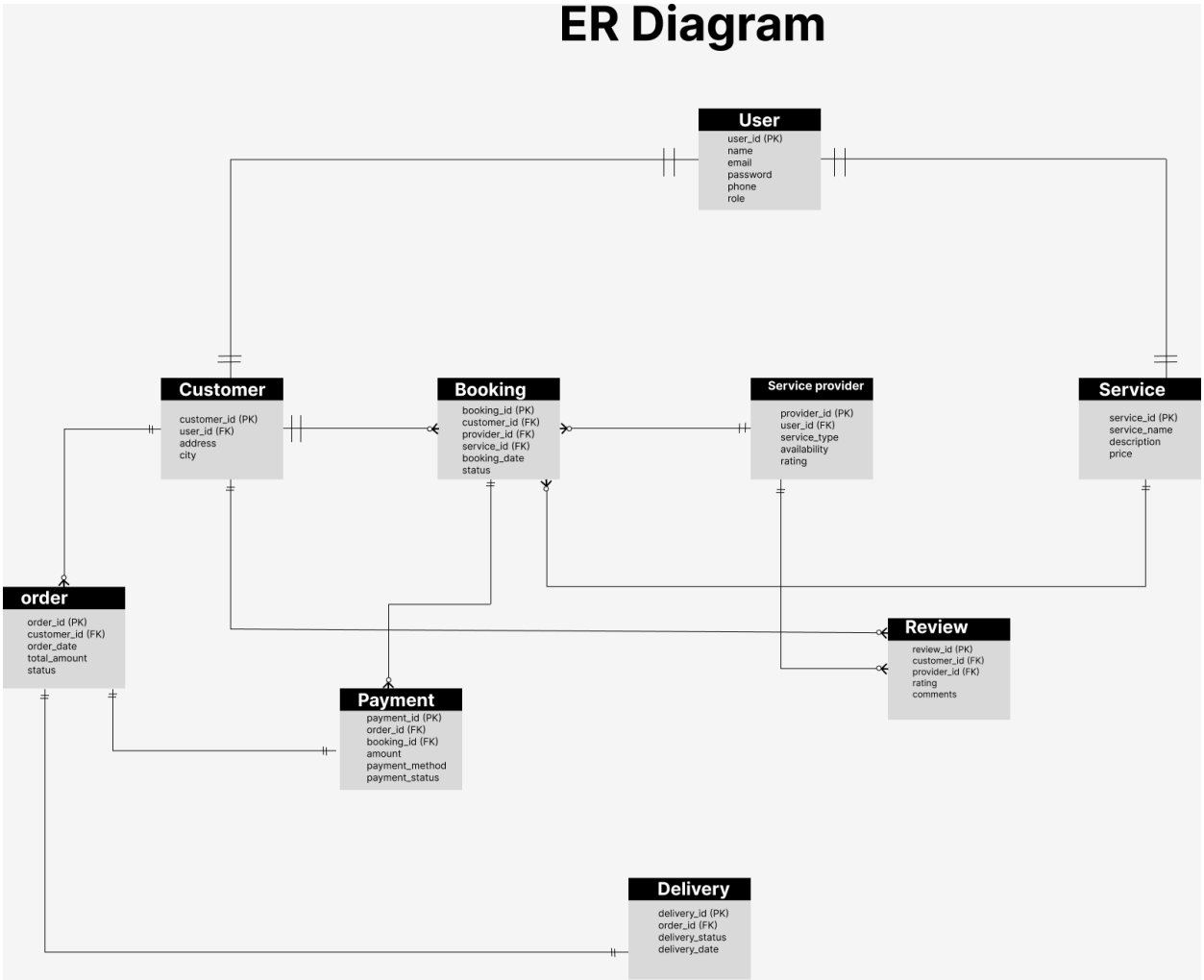

Figure 5.4 ER Diagram

https://www.figma.com/design/Qmvh7M0BxhY8t40uRD8Lu3/Untitled?node-id=2-201&t=LK0SNkUMbH1hLIA5-1

5.5 Data Flow Diagram

5.5.1 DFD level 0

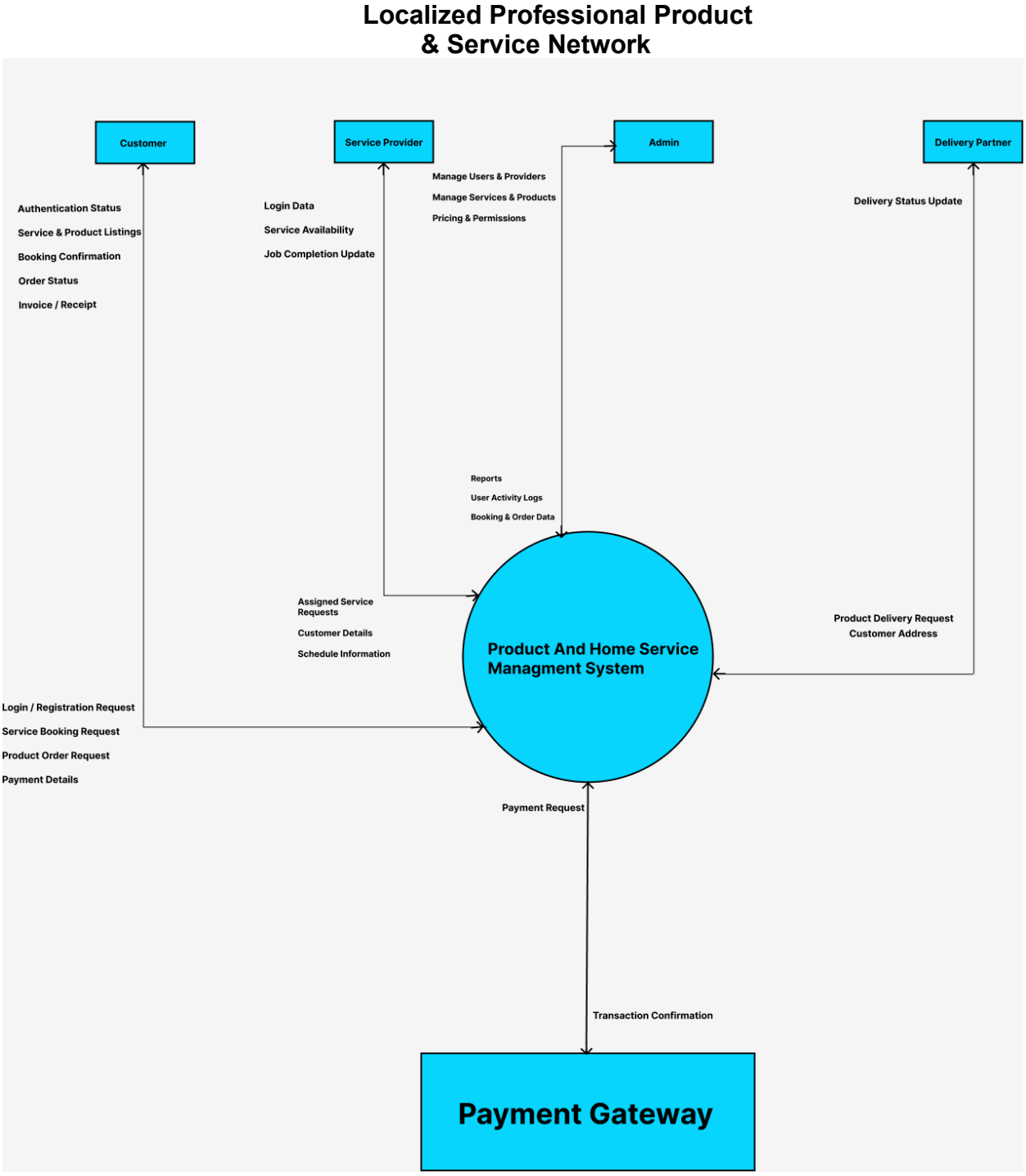

Figure 5.5.1 DFD Level 0

5.5.2 DFD level 1

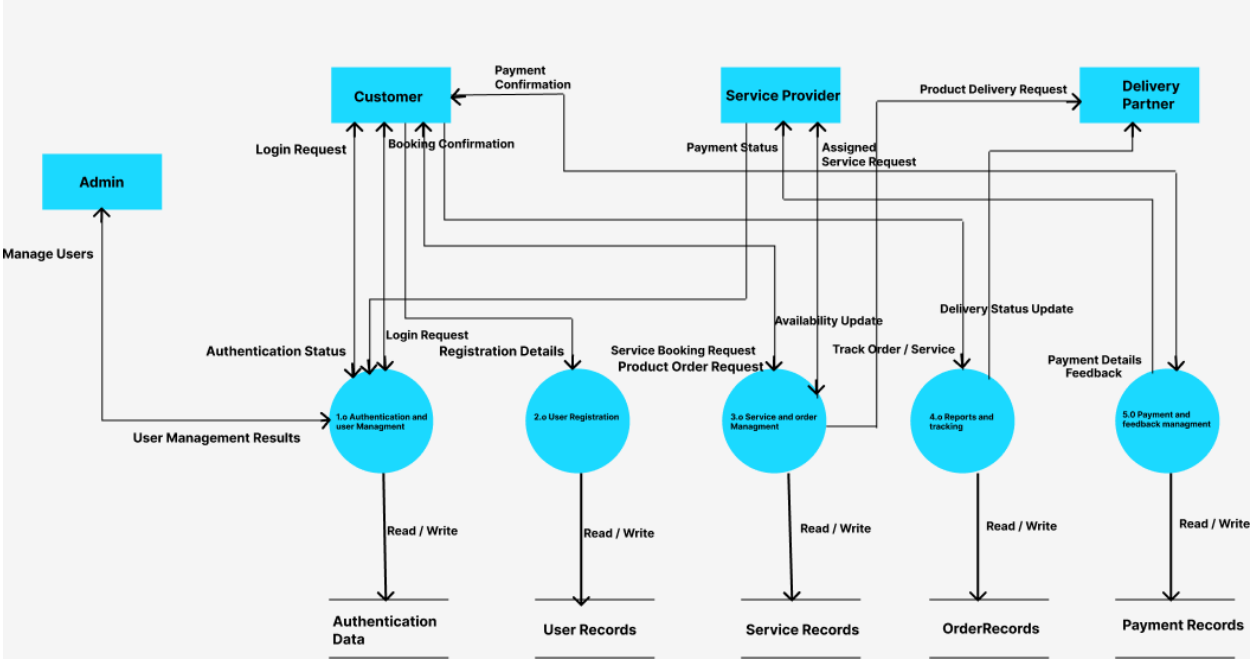

Figure 5.5.2 DFD Level 1

5.5.3 DFD Level 2

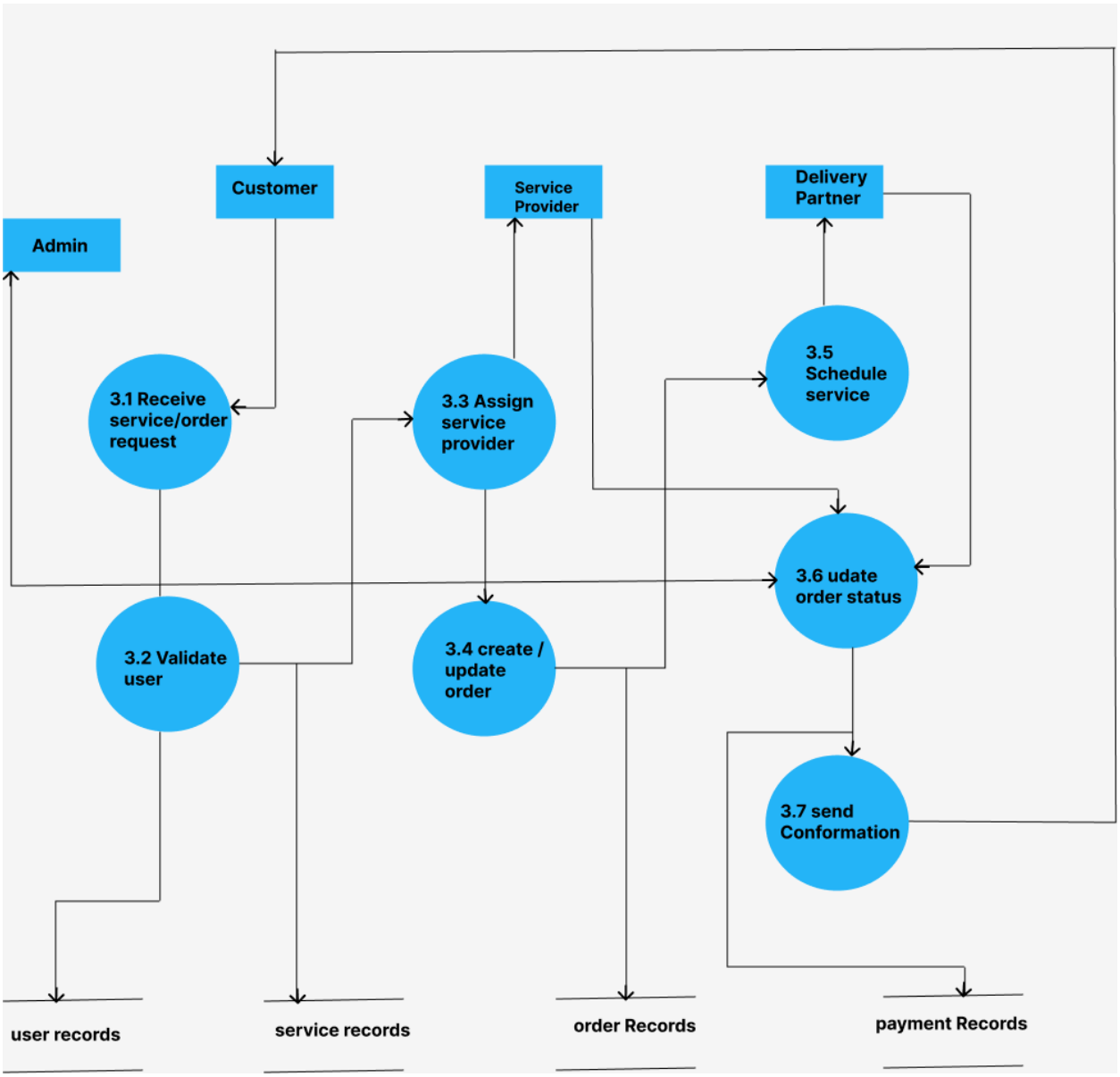

Figure 5.5.3 DFD Level 2

6 References

This section should provide a complete list of all documents referenced at specific point in time. Each document should be identified by title, report number (if applicable), date, and publishing organization. Specify the sources from which the references can be obtained (This section is like the bibliography in a published book).

Ref. No.	Document Title	Date of Release/ Publication	Document Source
PGBH01-2025-Proposal	Project Proposal	Oct 20, 2025	<Give the path of your Project repository/Folder>
PGBH01-2025-FS	Functional Specification	Oct 20, 2025	<Give the path of your Project repository/Folder>

7 Appendices

Include supporting detail that would be too distracting to include in the main body of the document.